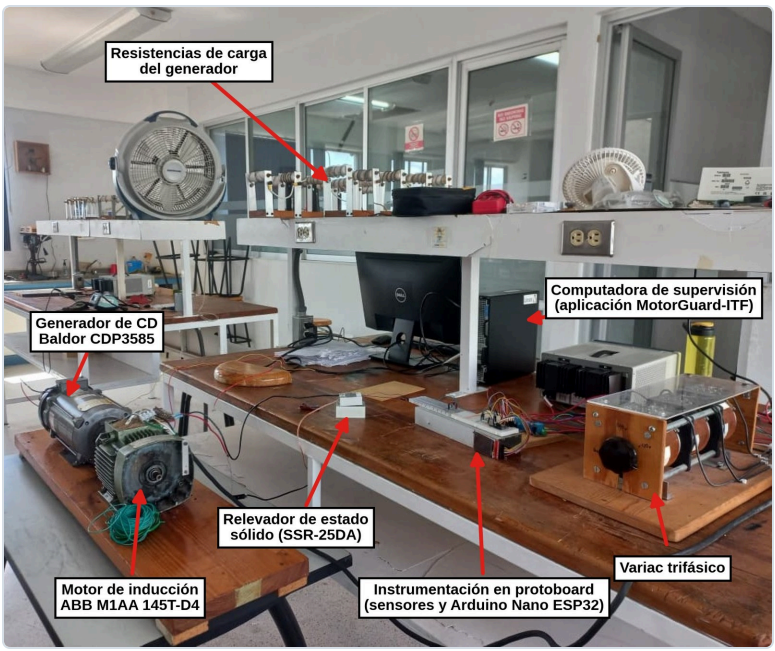
PORTAFOLIO · PROYECTO DE TESIS · UASLP 2025–2026

Detección de cortocircuitos entre espiras en motores de inducción

Juan Emmanuel Luna Velázquez
Egresado de Ingeniería Mecatrónica
Matehuala, S.L.P. · linkedin.com/in/juan-emmanuel-luna
juanemmanuel2220@gmail.com

Un cortocircuito entre espiras del devanado empieza pequeño y, si no se detecta, termina quemando el motor. Construí un sistema completo que lo detecta **midiendo solo las corrientes y voltajes de alimentación**, sin tocar el motor: desde el banco de pruebas y el firmware en tiempo real hasta el análisis con aprendizaje automático y una aplicación con asistente de IA local.

EL BANCO DE PRUEBAS



Banco en el laboratorio de la UAMRA, con sus partes señaladas.

1.5 kW
motor de inducción trifásico ABB, alimentado a 123 V por fase

0–75 %
de carga con un generador de CD y un banco de resistencias

3 severidades
de falla: 7.0, 14.3 y 21.6 % de las espiras de una fase

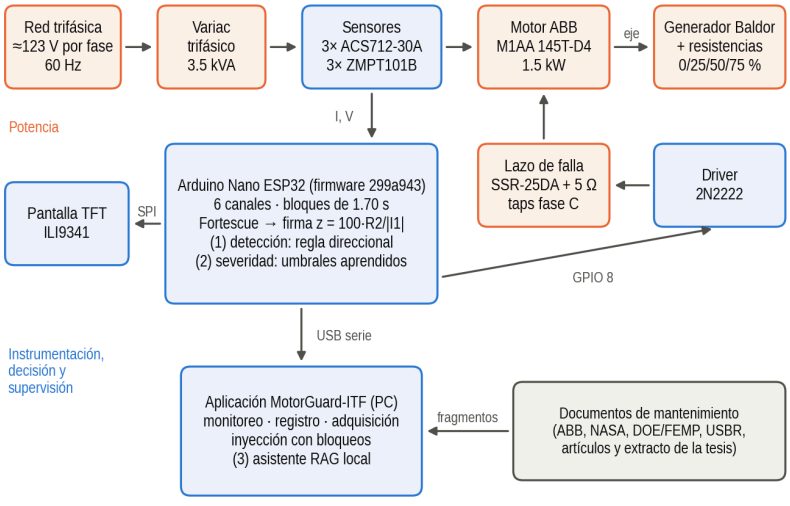
6 canales
3 sensores de corriente ACS712 y 3 de voltaje ZMPT101B

14 425
bloques procesados en 6.8 h seguidas sin perder ninguno

597
casos sano/falla capturados en el banco para entrenar y validar

La falla se emula con derivaciones del devanado y un relevador de estado sólido que la conecta en pulsos cortos, con un fusible y un resistor limitador de 5 Ω para proteger el motor.

CÓMO ESTÁ ARMADO EL SISTEMA



Potencia (naranja) e instrumentación, decisión y supervisión (azul). El ESP32 decide solo; la PC registra, controla la inyección de falla y consulta el asistente.

HERRAMIENTAS

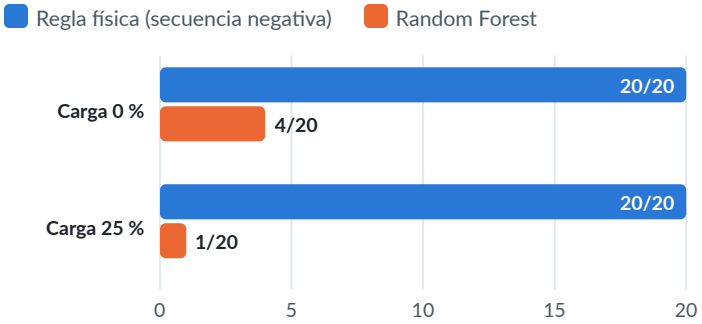
ESP32 · C/C++ | ACS712 · ZMPT101B | Componentes simétricas (Fortescue) | Python · scikit-learn | Estadística (Wilson, Poisson, McNemar) | PySide6 · pyqtgraph | llama.cpp · RAG | git · Linux · Docker · pytest

QUÉ HACE CADA PARTE

- **Firmware en C/C++ (ESP32):** muestrea 6 canales a 2 403.85 Hz en bloques de 4 096 muestras (1.70 s), calcula los fasores y la **corriente de secuencia negativa** con las componentes simétricas de Fortescue, y aplica una regla de detección con persistencia de 3 bloques y enclavamiento de la alarma. Muestra el estado en una pantalla TFT y envía telemetría por USB.
- **Análisis en Python:** dataset de 597 casos; Random Forest, Gradient Boosting, regresión logística y árbol de decisión con validación que deja fuera grupos completos (cargas no vistas). Protocolos registrados en git antes de analizar y validaciones ciegas.
- **Aplicación de escritorio (PySide6):** monitoreo en tiempo real, control seguro de la inyección de falla y un asistente RAG local (llama.cpp) que recomienda acciones citando manuales de mantenimiento de ABB, NASA, DOE y USBR.

RESULTADO PRINCIPAL: LA IA NO GENERALIZÓ, LA FÍSICA SÍ

Fallas leves (7.0 %) detectadas en la validación ciega, de 20 pulsos por carga

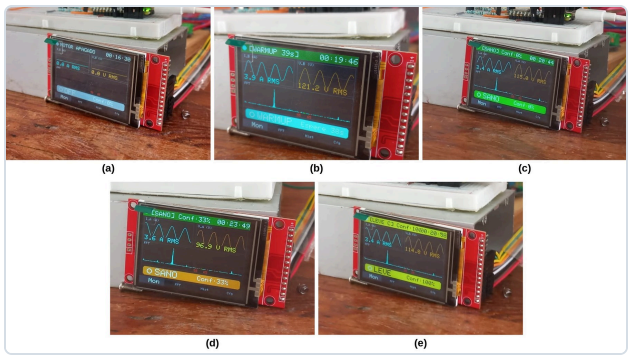


El Random Forest se veía muy bien en la validación cruzada y tuvo **menos falsas alarmas** (2 contra 21 de la regla en 4.9 h). Pero con cargas bajas casi no detectó las fallas leves: había aprendido un **atajo**, una característica que dependía de la carga y no de la falla.

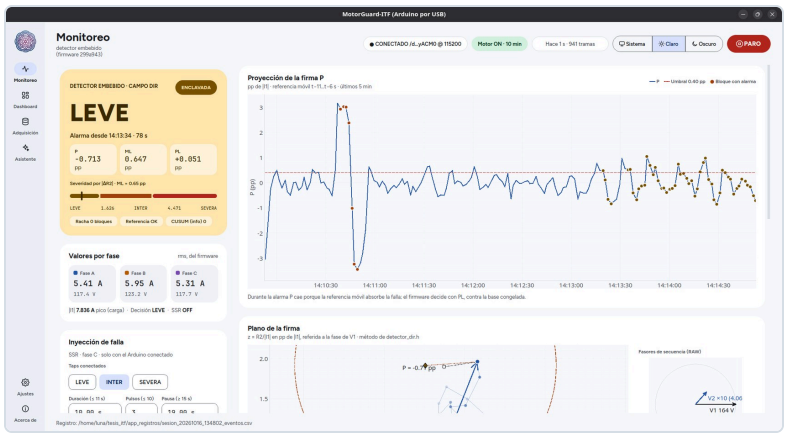
La regla física, basada en la corriente de secuencia negativa, detectó todas. Por eso rediseñé el sistema: **la detección la hace la regla** en el microcontrolador, y la IA se usa donde sí funcionó: el **clasificador de severidad acertó 596 de 597 casos** con cargas no vistas, y el asistente traduce la alarma en una recomendación.

Lo que no salió (y reporté): en una prueba larga de 90 min sin falla, la regla dio 7.3 falsas alarmas por hora (IC 95 %: 3.7–13.1), lejos de la meta de 1 por hora. La causa es ruido lento del propio banco; aumentar la persistencia la baja, pero hay que validarlo en una campaña nueva. Es el siguiente problema por resolver.

LO QUE VE EL OPERADOR



Pantalla TFT del ESP32: (a) motor apagado, (b) calentamiento, (c) y (d) motor sano, (e) falla leve detectada.



Aplicación de escritorio con una alarma LEVE enclavada: firma de la falla en el tiempo, valores por fase y control de la inyección.

MI PAPEL

- Armé y cableé el banco, hice las mediciones y las campañas de captura.
- Definí la regla de detección, los protocolos y los criterios de aceptación antes de ver los resultados.
- Programé el firmware, el análisis y la aplicación, apoyándome en asistentes de IA para acelerar el código; las decisiones técnicas y la validación fueron mías.
- Redacté la tesis (144 páginas) y un manuscrito en formato IEEE con mis asesores.

Lo que aprendí

- Validar con condiciones que el modelo no vio: una buena exactitud promedio puede esconder un atajo.
- Diseñar el banco pensando en la seguridad: fusible, resistor limitador y pulsos cortos.
- Reportar lo que no funciona con la misma claridad que lo que sí.